

CRC Package Interface Specification

Model-matrix, initialization, E-step, capture-model M-step, and outcome-likelihood implementation addendum

1 Model-matrix infrastructure

The package now converts parsed model specifications into complete capture-cell grids and aligned observation-level states. The infrastructure includes the capture and outcome design matrices, misclassification contributions, and a direct mapping from each observation state to its capture cell. Construction of these components is integrated into `crc_fit()`. The capture-model M-step is available as an internal fitting component, although the complete EM algorithm has not yet been integrated into `crc_fit()`.

1.1 Capture-model matrix

`build_capture_matrix()` constructs the Cartesian product of all capture histories, categorical levels used by the capture model, and latent classes. For misclassified variables, true-category levels and their ordering are obtained from the corresponding confusion matrices. The function returns:

- `cells`, the complete capture-model cell grid;
- `matrix`, the design matrix generated from the parsed terms;
- `unobserved`, an indicator of all-zero capture histories.

With J capture sources, K latent classes, and categorical variables with $C_1, \dots, C_p$ levels, the grid contains $2^J K \prod_{q=1}^p C_q$ rows.

1.2 Initial observation states

`build_observation_states()` repeats every observed unit once for each latent class. It retains the variables needed by the capture model, outcome model, and misclassification mechanisms. It also keeps aligned copies of the standardized outcome and observed values of misclassified variables. An observation identifier maps every expanded row back to its original unit. Only variables used by the capture or outcome model are retained in the state data. Recorded values of variables subject to misclassification are stored separately for evaluation of their confusion-matrix contributions.

1.3 Misclassified-variable expansion

`expand_misclass_states()` expands the current states over every possible true level of one misclassified variable. Each expanded row receives the contribution

$$\Pr(\tilde{G}_j = h \mid G_j = g)$$

from its confusion matrix. When `true_if` applies, the observed category receives probability one and all other candidate categories receive probability zero.

For several misclassified variables, separate confusion matrices are used and their contributions are multiplied. This corresponds to conditional independence of the misclassification mechanisms given their respective true categories. Dependent mechanisms may be supported in a future extension.

These contributions are likelihood terms, not posterior probabilities, and must not be normalized at this stage.

`expand_all_misclass_states()` applies this expansion successively to every specified variable. When no misclassification mechanism is supplied, it assigns a contribution of one to every latent state.

1.4 Outcome-model matrix

`build_outcome_matrix()` applies the parsed outcome terms to the fully expanded state data. Its rows therefore remain aligned with the state data, standardized outcomes, observation identifiers, and accumulated misclassification contributions. It returns `NULL` when no outcome model is specified.

Every predictor in `outcome_formula` is required to occur in `capture_formula`. This ensures that the predictor is represented in the complete capture-cell grid and is consequently available for prediction in the all-zero cells. The formulas may contain different terms; only the set of variables is constrained.

1.5 Capture-cell indices

`build_capture_index()` returns one integer index for every expanded observation state. Matching uses the capture indicators, categorical variables in the capture model, and latent class. If the i th value is r , row i of the state data uses row r of the capture-cell grid and capture design matrix. The vector avoids repeated joins during future EM iterations. Construction fails if any state cannot be matched to the grid.

1.6 Unified model-matrix object

`build_model_matrices()` coordinates the complete construction sequence and returns an object of class `crc_model_matrices` with:

- `capture`, containing the cell grid, capture design matrix, and all-zero-cell indicator;
- `states`, containing expanded state data and aligned metadata;
- `outcome_matrix`, containing the outcome design matrix or `NULL`;
- `capture_index`, linking state rows to capture-cell rows.

`crc_fit()` constructs this object after parsing and compatibility validation and stores it as `model_matrices`. The returned object remains in class `crcfit_unfitted` and has `fitted = FALSE`.

1.7 Cross-component validation

Compatibility checks now reject outcome predictors absent from the capture model. They also reject variables specified in `misclass` that are used by neither the capture nor the outcome model. This prevents redundant state expansion and makes the population-cell interpretation explicit.

2 Initialization

`initialize_crc()` creates normalized initial weights for the expanded observation states and expected counts for the complete capture-cell grid. For multiple latent classes, independent gamma draws produce a symmetric Dirichlet allocation within each observation and candidate true-category configuration. These allocations are multiplied by the misclassification contributions and normalized within each observed unit. The concentration is set by `control$init_alpha` and defaults to 20.

State weights are aggregated through the precomputed capture-cell indices. Observed cells receive the corresponding weighted counts, while every all-zero capture cell receives an initial expected count of one. The resulting `crc_initialization` object stores the state weights, cell counts, and Dirichlet concentration and is retained by `crc_fit()`.

3 Control and progress interface

`parse_control()` resolves the user-supplied `control` list against complete defaults and returns an object of class `crc_control`. The resolved object is stored by `crc_fit()`. Partial overrides are supported, while unknown, duplicated, unnamed, and invalid settings are rejected. The available settings are:

- `init_alpha`, the positive Dirichlet concentration, with default 20;
- `em$max_iter` and `em$tolerance`, with defaults 1000 and 10^{-6} ;
- `capture$max_iter` and `capture$tolerance`, with defaults 100 and 10^{-8} ;
- `outcome$max_iter` and `outcome$relative_tolerance`, with defaults 1000 and 10^{-8} .

The separate logical argument `verbose` controls user-facing progress information and defaults to `FALSE`. It is validated and stored by `crc_fit()`; optimizer-specific tracing is not part of the public control interface.

4 Expectation step

`expectation_crc()` implements the distribution-independent part of the E-step. It accepts one fitted capture-model mean for every row of the complete capture-cell grid and, when an outcome model is present, one outcome log-likelihood contribution for every expanded observation state. A missing outcome model is represented by zero log-likelihood contributions, equivalent to likelihood contributions of one.

For state (i, g, k) , the unnormalized log weight is

$$\log m_{Y_i, g, k} + \log \Pr(\tilde{G}_i \mid G_i = g) + \log L_V(i \mid g).$$

Exact zero misclassification probabilities remain negative infinity on the log scale and therefore produce posterior weights of exactly zero. The function verifies that each observed unit has at least one admissible state and normalizes the weights within unit after subtracting the largest log weight. This log-sum-exp calculation avoids unnecessary underflow and overflow.

The normalized state weights are aggregated through `capture_index` to update the expected counts for observed capture histories. Expected counts for all-zero histories are set to their current fitted capture-model means. The function returns a `crc_estep` object containing one posterior weight per

expanded state and one expected count per capture cell. It is not yet called by `crc_fit()`, because the outcome M-step and the complete EM iteration have not been implemented.

5 Capture-model maximization

`maximize_capture_crc()` implements the capture component of the M-step. Given expected capture-cell counts n_c and capture-model rows x_c , it maximizes

$$Q_Y(\theta) = \sum_c \{n_c \log m_c(\theta) - m_c(\theta)\}, \quad m_c(\theta) = \exp(x_c^\top \theta).$$

The maximization is performed with `glm.fit()` using a Poisson family and logarithmic link. Fractional expected cell counts are valid for this objective because the omitted terms do not depend on θ . The specific Poisson warning about non-integer counts is suppressed, while other warnings remain visible.

When no starting values are supplied, all coefficients start at zero, giving initial fitted cell means of one. The function also accepts previous coefficients as warm starts for later EM iterations. The public capture controls will be translated to `glm.control()` when the EM loop is integrated.

The function rejects invalid counts or starting values, non-convergence, rank-deficient design matrices, non-finite coefficients, and non-positive or non-finite fitted means. It returns a `crc_capture_mstep` object with the coefficients, fitted cell means, maximized objective value, iteration count, and convergence indicator.

6 Outcome log-likelihood

`outcome_loglik_crc()` evaluates one outcome log-likelihood contribution for every expanded observation state. Its arguments are the aligned standardized outcome table, a positive mean `mu` for every state, the outcome distribution, and, for the negative binomial model, a single positive dispersion parameter `sigma`. Predictors in `outcome_formula`, including `.latent`, may therefore produce different means for different candidate states of the same observed unit.

Let W denote the corresponding untruncated count. Under "ztpois", $W \sim \text{Poisson}(\mu)$. Under "ztnegbin", W has mean μ , size $1/\sigma$, and variance $\mu + \sigma\mu^2$. In both cases the modeled outcome is the conditional variable $V = W \mid W \geq 1$. If f , F , and $S(q) = \Pr(W > q)$ denote the untruncated mass, distribution, and survival functions, respectively, the implemented contribution is

$$\ell_V = \begin{cases} \log f(v) - \log S(0), & \text{exact observation,} \\ \log\{S(l-1) - S(u)\} - \log S(0), & \text{interval } l \leq V \leq u, \\ \log S(l-1) - \log S(0), & \text{right censoring } V \geq l, \\ 0, & \text{missing outcome.} \end{cases}$$

Thus, interval bounds are inclusive and missing outcomes contribute a likelihood factor of one under the ignorable coarsening assumption. Exact contributions use the log probability mass directly. Interval contributions use upper-tail log probabilities and `log_sub_crc()`, avoiding unstable subtraction of nearly equal ordinary probabilities in the right tail. No artificial probability floor is applied.

6.1 Log-scale probability subtraction

`log_sub_crc()` provides the stable operation

$$\log\{\exp(a) - \exp(b)\}, \quad a \geq b,$$

using `expm1()`. This helper is used for interval-censored outcome likelihoods, where the required probability is obtained by subtracting two survival probabilities. It also handles the case in which both probabilities are zero and their log values are negative infinity.

7 Planned outcome-model maximization

The outcome component of the M-step will maximize the posterior-weighted expected log-likelihood

$$Q_V(\phi) = \sum_s \tau_s \ell_V(s; \phi),$$

where s indexes the expanded observation states, τ_s is the current posterior state weight, and $\ell_V(s; \phi)$ is provided by `outcome_loglik_crc()`. Missing outcomes have zero log-likelihood contributions and therefore do not affect this maximization.

The implementation will minimize $-Q_V$ with `optim()` using `method = "BFGS"`. For `"ztpois"`, the parameter vector will contain only the outcome regression coefficients β , with $\mu_s = \exp(z_s^T \beta)$. For `"ztnegbin"`, it will also contain `log_sigma`; optimizing the logarithm guarantees $\sigma > 0$ without box constraints. No arbitrary coefficient bounds or user-selected optimizer will be exposed initially. Numerical derivatives will be used in the first implementation; analytical derivatives may be added if profiling demonstrates a material need.

For the first M-step, preliminary regression coefficients will be obtained from a posterior-weighted Poisson fit to exact outcomes. If there are too few exact outcomes for that fit, the intercept will be initialized as the logarithm of the weighted mean of informative lower bounds, and the remaining coefficients will be set to zero. For the negative binomial model, a positive method-of-moments dispersion estimate based on exact outcomes will be used when available, with 0.25 as the conservative fallback. These quantities are starting values only and do not replace maximization of the zero-truncated, censored objective.

After the first iteration, the preceding outcome coefficients and `log_sigma`, when applicable, will be supplied as warm starts. The `max_iter` and `relative_tolerance` elements of `control$outcome` will be translated to the corresponding `optim()` controls.

An update will be accepted only when `optim()` reports convergence code zero, all parameters and the objective are finite, and the relevant Q_V value has not decreased beyond a small numerical tolerance. A failed optimization will be retried once from the conservative first-iteration values. If the retry also fails, fitting will stop with an informative error rather than silently retaining stale parameters.

8 Automated tests

The model-matrix, initialization, control, E-step, capture-model M-step, and outcome-likelihood infrastructure is covered by automated tests implemented with the `tinytest` package. Separate test

files cover model matrices, misclassification expansion, cross-component compatibility, initialization, control parsing, expectation calculations, capture-model maximization, and outcome log-likelihood calculations.

The existing tests cover capture grids and design matrices, row alignment, factor levels, all-zero cells, misclassification expansion, compatibility, initialization, and E-step behavior. Matrix, compatibility, and initialization tests exercise the implemented behavior through `crc_fit()`; the E-step is tested directly because it is not yet integrated into the fitting loop.

Initialization tests verify normalized, reproducible state weights and their capture-cell aggregation, including the single-class case and invalid control values. Fifteen E-step assertions cover posterior normalization, impossible states, cell aggregation, numerical stability, dimensions, and validation.

Thirteen control assertions cover defaults, partial nested overrides, the `verbose` argument, unknown settings, malformed subsections, and invalid values. Twelve capture-maximization assertions cover fitted-object structure, dimensions, finite coefficients and means, objective evaluation, warm starts, invalid inputs, and rank deficiency.

The outcome-likelihood tests compare the Poisson and negative binomial results with direct probability calculations for mixed exact, interval-censored, right-censored, and missing outcomes. They also verify inclusive interval bounds, a right-censoring boundary at one, numerical stability in the far right tail, and validation of the mean and dispersion inputs. The complete suite now contains 91 passing assertions, of which nine exercise `outcome_loglik_crc()`.

9 Required next steps

The model-matrix, initialization, control, distribution-independent E-step, capture-model M-step, and outcome-likelihood infrastructure and their automated tests are complete at the design level. Development should continue in the following order.

1. Implement and test the outcome-model M-step according to the planned optimization and warm-start strategy above.
2. Integrate initialization, the capture and outcome M-steps, outcome likelihood evaluation, and `expectation_crc()` into the EM iteration, using previous estimates as warm starts.
3. Add convergence controls, numerical safeguards, and diagnostics before exposing fitted estimates through user-facing methods.

EM updates must follow the methodology in the accompanying paper and simulation implementation. The existing automated tests establish the initialization, matrix, and alignment invariants on which those iterative components depend.